

Università degli studi di Napoli Parthenope

Progetto Programmazione 3

A.A. 2023/2024

Guardia e Ladro 1

Proponenti:

Porritiello Francesco 0124002486

Esposito Francesco 0124002629

Sommario

Traccia progetto 3

Intelligenza Artificiale 5

- **Factory Method** 11
- **Observer** 12
- **Command** 13

Traccia progetto

Si vuole sviluppare un'applicazione per la simulazione di un gioco denominato guardia e ladro. Un utente (giocatore) si identifica tramite il nome e cognome. Si suppone di avere una stanza pavimentata a tasselli quadrati (caselle), dotata di pareti esterne ed interne. Nella stanza sono presenti una guardia e un ladro. Il giocatore deve permettere al ladro di uscire dalla stanza. Sia il ladro che la guardia si muovono di una casella alla volta tra le otto caselle vicine.

Devono essere previsti diversi scenari. Per ogni scenario le strategie con cui si muove la guardia sono (K è la probabilità):

- nel K% dei casi la guardia si muove a caso in una delle otto caselle vicine possibili (parete permettendo);
- nel (100 − K)% dei casi la direzione della guardia viene calcolata usando l'algoritmo di ottimizzazione ant colony.

Inoltre, nella stanza ci sono oggetti di diversi colori a disposizione del ladro:

- verde - la guardia va nella direzione opposta a quella del ladro per 10 secondi.
- giallo - la guardia va in una direzione casuale per 10 secondi.
- rosso - la guardia va nella direzione dell'uscita (ottimizzazione ant colony).

Scrivere un programma per la gestione del gioco che permette di visualizzare, ad ogni inizio e fine partita, la classifica dei risultati migliori ottenuti, da tutti i giocatori, in tutte le partite (minore numero di passi per raggiungere l'uscita).

Introduzione

Il progetto è stato sviluppato mediante l'uso di libreria Swing per il supporto grafico.

La logica relativa al movimento della guardia si trova nella classe Strategy, essa funge da interfaccia definendo un metodo move().

Ogni classe che implementa Strategy esegue un override della firma del metodo move() per offrire specifiche implementazioni dei movimenti della guardia a seconda del contesto.

aco_move() {
} : gestisce il movimento della guardia attraverso Ant Colony Optimization.

green_move() {
} : gestisce il movimento della guardia in modo speculare rispetto al ladro quando il ladro è su una casella verde.

rand_move() {
} : gestisce il movimento della guardia in modo randomico quando il ladro si trova su una casella gialla.

In ognuno di questi approcci viene fatto muovere di una casella alla volta la guardia nelle 8 caselle adiacenti.

Intelligenza Artificiale

L’algoritmo di Ant Colony Optimization è un algoritmo del tipo Swarm Intelligence. L’algoritmo fu introdotto per la prima volta nel 1992 da Marco Dorigo del Politecnico di Milano. Esso viene utilizzato per la risoluzione di problemi computazionali complessi. L’idea di base di questo algoritmo prende spunto dall’organizzazione di una colonia di formiche, si usa infatti un meccanismo di feedback positivo come una sorta di feromone virtuale, per rafforzare quelle parti di soluzione che contribuiscono alla risoluzione del problema. Per evitare invece la convergenza verso opzioni non idonee, viene utilizzato un meccanismo di feedback negativo (ad esempio l’evaporizzazione del feromone virtuale) che introduce una componente temporale nell’algoritmo. Il punto di forza dell’algoritmo consiste quindi nella creazione di un sistema intelligente distribuito.

Implementazione dell’ACO

L’Ant Colony Optimization è stato adattato al nostro progetto per risolvere un problema diverso da quello a cui si presta solitamente, cioè il problema del commesso viaggiatore.

In questo caso non abbiamo delle città rappresentate da nodi collegati da archi cioè un grafo, ma abbiamo punti all’interno di una matrice che sono in continuo movimento.

Vogliamo quindi trovare ad ogni variazione della posizione di A e B il percorso più breve tra di essi considerando anche gli ostacoli quali i muri.

La prima fase dell’algoritmo consiste in una fase di esplorazione, vengono quindi inizializzate le formiche nella stanza e il feromone sul suolo.

Le formiche vengono fatte successivamente muovere in tutta la stanza di una casella alla volta nelle 8 caselle adiacenti in modo casuale alla ricerca del ladro.

Quando il ladro viene trovato la formica interessata rilascia del feromone sul suolo per marcare la posizione del ladro.

```
private void ants_move() {
    init_ants_random();
    boolean targetFound = false;
    Point target = new Point();
    if(isRed) {
        Point exit = new Point(room.exit[rand.nextInt( bound: 3)]);
        target.setLocation(exit);
    }
    else {
        target.setLocation(thief);
    }
    while (!targetFound) {
        for (int i = 0; i < ANTS; i++) {
            Point point = new Point();
            do {
                int[] direction = ANTS_DIRECTIONS[rand.nextInt( bound: 8)];
                point.setLocation( x: direction[0] + ants[i].x,  y: direction[1] + ants[i].y);
            } while (!is_valid(point));

            ants[i].setLocation(point.x, point.y);

            if (ants[i].equals(target)) {
                targetFound = true;
                latest_seen.setLocation(ants[i]);
                drop_pheromones(ants[i]);
                break;
            }
        }
    }
    find_path();
    pheromones_evaporation();
}
```

Nella seconda fase inizializziamo tutte le formiche alla colonia cioè nella posizione corrente della guardia perchè vogliamo calcolare il percorso più breve dalla guardia all'ultima posizione in cui è stato avvistato il ladro.

Ogni singola formica viene fatta muovere di una posizione alla volta ed ogni posizione va a definire una parte del percorso da essa intrapresa.

Una volta iterato su tutte le formiche per le iterazioni definite tra tutti i path generati da ognuna di essa viene scelto quello più breve, cioè, con meno passi per arrivare a destinazione, su questo percorso vengono poi depositate tracce di feromoni.

Ogni volta che avviene uno spostamento del ladro ne consegue un movimento della guardia di un passo verso l'ultima posizione conosciuta ed un ricalcolo allo stesso tempo del nuovo path più breve per arrivare alla sua nuova posizione. Qui entra in gioco l'evaporazione del feromone così da far prediligere tracce di feromone più fresche in modo da arrestare il suo movimento verso un vecchio percorso.

Infine la guardia effettuerà le scelte dei suoi successivi spostamenti analizzando le informazioni intorno a se stesso valutando quantità di feromoni presenti e visibilità.

La casella intorno al ladro con il miglior score sarà il prossimo movimento intrapreso dalla guardia.

```
    private void find_path() {
        init_ants_to_guard();
        int ANTS_ITERATION = 20;
        List<Point> shortestPath = null;
        for (int j = 0; j < ANTS_ITERATION; j++) {
            for (int i = 0; i < ANTS; i++) {
                Point current_point = new Point(ants[i]);
                List<Point> path = new ArrayList<>();
                while (!isTarget(current_point)) {
                    Point next_point;
                    do {
                        int[] dir = ANTS_DIRECTIONS[rand.nextInt( bound: 8)];
                        next_point = new Point( x: current_point.x + dir[0],  y: current_point.y + dir[1]);
                    } while (!is_valid(next_point));
                    path.add(next_point);
                    current_point = next_point;
                }
                if (shortestPath == null || path.size() < shortestPath.size()) {
                    shortestPath = new ArrayList<>(path);
                }
            }
        }
        assert shortestPath != null;
        for (Point step : shortestPath) {
            drop_pheromones(step);
        }
    }

}
```

Green Move

In questo metodo prendiamo le coordinate della direzione che ne risulta dallo spostamento del ladro, le neghiamo e le sommiamo con le coordinate della posizione corrente della guardia.

In questo modo faremo muovere la guardia in modo opposto al ladro.

```
class green_move implements Strategy {
    4 usages
    private final int guardX;
    4 usages
    private final int guardY;

    2 usages  Francesco
    public green_move(int guardX, int guardY) {
        this.guardX = guardX;
        this.guardY = guardY;
    }

    5 usages  Francesco +1 *
    @Override
    public int[] move(int[] thief_direction) {
        Room room = new Room();
        int x = thief_direction[0];
        int y = thief_direction[1];
        int[] guard_movement = new int[]{-x, -y};
        if (guardX + guard_movement[0] < 0 || guardX + guard_movement[0] >= room.row ||
                guardY + guard_movement[1] < 0 || guardY + guard_movement[1] >= room.column) {
            return new int[]{0, 0};
        }
        if (room.matrix[guardX + guard_movement[0]][guardY + guard_movement[1]] == Color.BLACK) {
            return new int[]{0, 0};
        }
        return guard_movement;
    }
}
```

Rand Move

In questo metodo vengono scelte delle coordinate che rappresentano le direzioni delle 8 caselle adiacenti in modo casuale, vengono successivamente sommate poi alle coordinate della posizione corrente della guardia.



```
class rand_move implements Strategy {
    1 usage   Francesco
    private boolean isExit(Point[] exit, int x, int y) {
        for (Point p : exit) {
            if (p.x == x && p.y == y) {
                return true;
            }
        }
        return false;
    }
    1 usage   Francesco
    private int[] rand_adjacent_8(int[] current_pos) {
        Room room = new Room();
        final int[][] DIRECTIONS = {{-1,0},{1,0},{0,-1},{0,1},{1,1},{-1,-1},{1,-1},{-1,1}};

        Random random = new Random();
        int randIndex;
        int[] direction;
        int newX, newY;

        do {
            randIndex = random.nextInt( bound: 8);
            direction = DIRECTIONS[randIndex];
            newX = current_pos[0] + direction[0];
            newY = current_pos[1] + direction[1];
        } while (newX < 0 || newX >= room.row || newY < 0 || newY >= room.column || room.matrix[newX][newY] == Color.BLACK || isExit(room.exit, newX, newY));

        current_pos[0] = newX;
        current_pos[1] = newY;

        return current_pos;
    }
```

Pattern utilizzati

Ecco, di seguito, l’elenco dei pattern utilizzati:

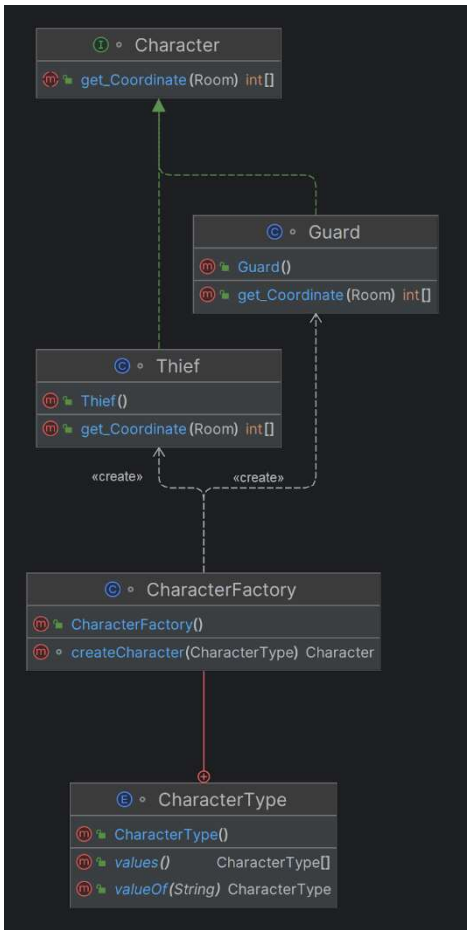
- Factory Method
- Observer
- Command
- Strategy

Factory Method

L'interfaccia Character dichiara un metodo get_Coordinate che rappresenta un comportamento comune per ottenere le coordinate di un personaggio.
Le classi Thief e Guard implementano l'interfaccia Character, fornendo implementazioni specifiche per il metodo get_Coordinate. Queste classi rappresentano i personaggi del gioco.

La classe CharacterFactory è responsabile della creazione degli oggetti Character. Il metodo createCharacter è il Factory Method che restituisce un'istanza di una sottoclasse di Character in base al tipo specificato. In questo caso, può creare un Thief o un Guard.
Il Factory Method è utilizzato nel metodo main della classe Main per creare un Thief e un Guard.
Qui, il Factory Method ti consente di creare dinamicamente un Thief e un Guard senza dover conoscere le implementazioni specifiche, seguendo il principio dell'incapsulamento.

Il Factory Method permette di delegare la creazione di oggetti a sottoclassi, permettendo la flessibilità nell'istanziare diverse implementazioni di un'interfaccia o classe astratta.

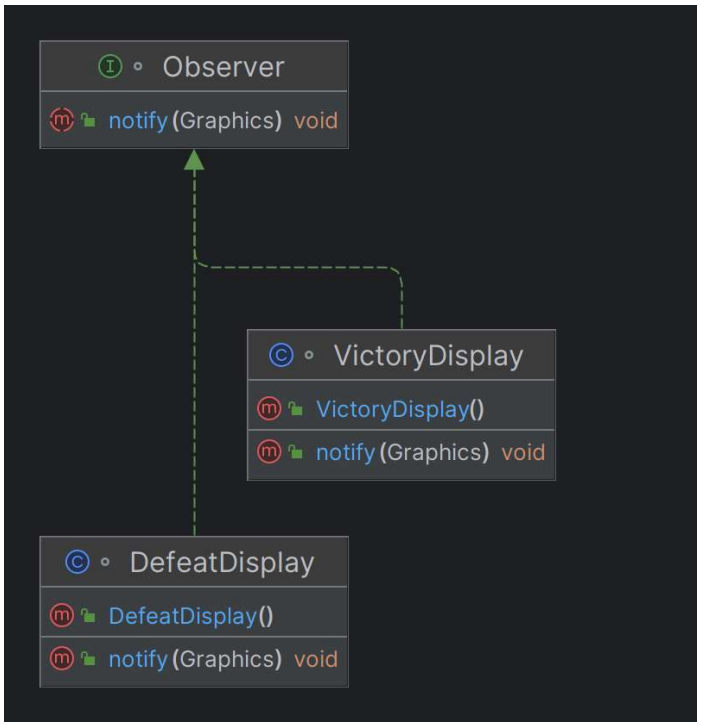


Observer

Il pattern Observer viene utilizzato per notificare gli oggetti interessati (osservatori) quando avviene un evento significativo, ovvero quando il giocatore vince o perde la partita.

L'interfaccia Observer definisce un metodo notify che viene chiamato quando l'evento rilevante si verifica.
VictoryDisplay e DefeatDisplay sono classi che implementano l'interfaccia Observer. Queste classi forniscono l'implementazione specifica del metodo notify per gestire la visualizzazione di un messaggio di vittoria o sconfitta.
La classe Game estende JFrame e funge da oggetto osservabile. Contiene una lista di osservatori (victoryObservers) e metodi per aggiungere osservatori (addObserver), notificare gli osservatori (notifyVictoryObservers), e gestire la logica del gioco.
Nel metodo handleKeyPress della classe Game, quando il giocatore vince o perde, viene chiamato notifyVictoryObservers, che a sua volta notifica tutti gli osservatori registrati, chiamando il metodo notify sull'interfaccia Observer.

Questo approccio consente di separare la logica di gioco dalla visualizzazione, consentendo a nuovi elementi di interfaccia utente di essere aggiunti in modo flessibile senza modificare la logica di gioco principale.
Gli oggetti di visualizzazione (VictoryDisplay e DefeatDisplay) sono gli osservatori che vengono notificati quando avviene un evento rilevante.



Command

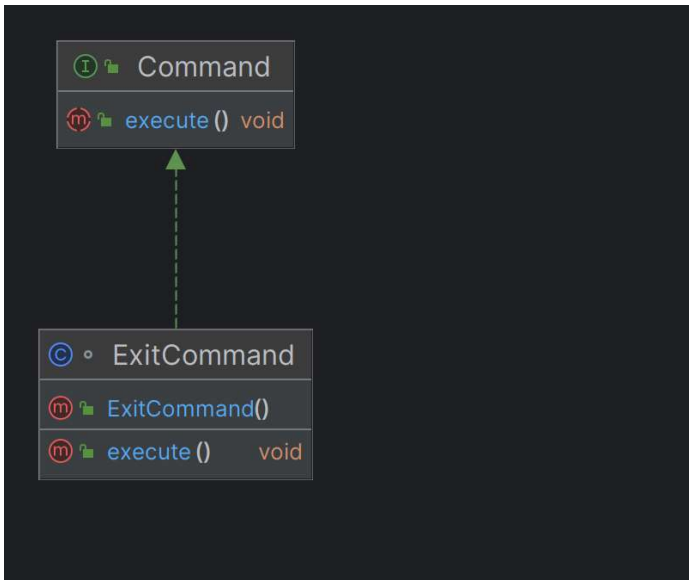
Il pattern Command è un design pattern comportamentale che consente di incapsulare una richiesta come un oggetto, consentendo così di parametrizzare i client con richieste diverse, accodare richieste e supportare operazioni annullabili.
L'interfaccia Command dichiara un metodo execute che rappresenta l'azione da eseguire. Le classi che implementano questa interfaccia forniranno l'implementazione specifica per l'azione associata.

La classe ExitCommand implementa l'interfaccia Command e fornisce un'implementazione specifica per l'azione di uscita.

Nel metodo main della classe Main, abbiamo creato un'istanza di ExitCommand ed e' assegnata come comando di uscita per il gioco.

In questo modo, quando viene premuto il tasto ESC durante il gioco, l'oggetto ExitCommand viene eseguito tramite il metodo execute, che termina il programma.
Nel metodo handleKeyPress della classe Game, se viene rilevata la pressione del tasto ESC, il metodo execute del comando di uscita viene chiamato.

In sintesi, il pattern Command consente di incapsulare richieste come oggetti, consentendo la parametrizzazione di oggetti client con operazioni diverse e supportando operazioni annullabili. Nel nostro caso, ExitCommand incapsula l'azione di uscita dal gioco.



Strategy

Il pattern Strategy è utilizzato per definire algoritmi di movimento dinamici per la guardia all'interno del gioco. In questo contesto, l'interfaccia Strategy rappresenta il comportamento di movimento della guardia, mentre le classi concrete (rand_move, green_move, aco_move) forniscono implementazioni specifiche di tali comportamenti.

L'interfaccia Strategy dichiara un metodo move, che accetta la configurazione della stanza e le coordinate del ladro come argomenti e restituisce le nuove coordinate della guardia.

